Centro Federal de Educação Tecnológica de Minas Gerais

Disciplina: Algoritmos e Estruturas de Dados
Professor: Fernando Garcia (fernandogdcf@cefetmg.br)

Prova 2

Aluno:_____ Nota: _____

1. (5,0 pontos) Considere a função abaixo que imprime os elementos de uma árvore binária:

```
void imprime(No *raiz);
{
   if (raiz != NULL) {
      imprime (raiz->esq);
      printf("%d ", raiz->chave);
      imprime (raiz->dir);
   }
}
```

Considere que os elementos foram inseridos nesta ordem: 1, 3, 7, 2, 5, 6, 4, 8. Escreva a ordem correta em que os elementos serão exibidos na tela pela função acima.

2. (6,0 pontos) Considere as seguintes declarações:

```
typedef struct No {          // Declaração do nó da Árvore
   int chave;                // Componente chave
   struct No* esq;           // Ponteiro para o elemento da esquerda
   struct No* dir;           // Ponteiro para o elemento da direita
} No;
No* raiz;                    // Ponteiro para a raiz da Árvore
// Operações
void criaArvore();           // Inicializa a árvore vazia
int pesquisa(int chave);     // Pesquisa por um nó na árvore
void insere(int chave);      // Insere novo nó na árvore
int remove(int chave);       // Remove um nó da árvore
void imprime();              // Imprime toda a árvore
```

Implemente uma função recursiva para o cálculo do menor elemento de uma árvore binária, isto é:

int calculaMenorElemento(No *raiz);

Considere que a árvore não esteja vazia. A árvore deve permanecer inalterada após a execução da função.

3. (7,0 pontos) Considere uma árvore AVL inicialmente vazia. Desenhe a árvore resultante após todas as operações indicadas:

a) Insira os seguintes valores, nesta ordem:
16, 43, 18, 47, 59, 32, 1, 12, 5

b) Partindo da árvore anterior, realize as seguintes remoções, nesta ordem:
47, 1, 43, 32, 18, 5

Observações:
- Na remoção de um nó com dois filhos, utilize seu sucessor em ordem (menor valor da subárvore direita) para substituí-lo.
- Após cada operação, verifique se a árvore continua balanceada.
- Não serão consideradas suficientes respostas que apresentem apenas a árvore final, sem demonstrar as configurações intermediárias após cada remoção.

4. (8,0 pontos) Escreva uma função que retorna o valor 1 se uma árvore binária é um heap e 0 caso contrário.

5. (9,0 pontos) Considere duas implementações de uma fila de prioridade:
- **Implementação A:** Max-Heap armazenado em um vetor.
- **Implementação B:** Lista simplesmente encadeada não ordenada.

a) Em relação ao número de comparações necessárias, qual implementação é mais eficiente na inserção de um elemento? Justifique sua resposta.

b) Em relação ao número de comparações necessárias, qual implementação é mais eficiente na remoção do elemento prioritário? Justifique sua resposta.

c) Em uma aplicação que realiza frequentemente inserções e remoções de elementos da fila de prioridade, qual das duas implementações tende a apresentar melhor desempenho? Justifique considerando o custo computacional das operações.

d) Em qual situação a lista encadeada não ordenada poderia ser uma alternativa interessante ao Max-Heap?